Chapter 8. HTTP Load Balancing with Ingress

A critical part of any application is getting network traffic to and from that application. As described in Chapter 7, Kubernetes has a set of capabilities to enable services to be exposed outside of the cluster. For many users and simple use cases, these capabilities are sufficient.

But the Service object operates at Layer 4 (according to the OSI model).[1] This means that it only forwards TCP and UDP connections and doesn't look inside of those connections. Because of this, hosting many applications on a cluster uses many different exposed services. In the case where these services are `type: NodePort`, you'll have to have clients connect to a unique port per service. In the case where these services are `type: LoadBalancer`, you'll be allocating (often expensive or scarce) cloud resources for each service. But for HTTP (Layer 7)-based services, we can do better.

When solving a similar problem in non-Kubernetes situations, users often turn to the idea of "virtual hosting." This is a mechanism to host many HTTP sites on a single IP address. Typically, the user uses a load balancer or reverse proxy to accept incoming connections on HTTP (80) and HTTPS (443) ports. That program then parses the HTTP connection and, based on the `Host` header and the URL path that is requested, proxies the HTTP call to some other program. In this way, that load balancer or reverse proxy directs traffic for decoding and directing incoming connections to the right "upstream" server.

Kubernetes calls its HTTP-based load-balancing system *Ingress*. Ingress is a Kubernetes-native way to implement the "virtual hosting" pattern we just discussed. One of the more complex aspects of the pattern is that the user has to manage the load balancer configuration file. In a dynamic environment and as the set of virtual hosts expands, this can be very complex. The Kubernetes Ingress system works to simplify this by (a) standardizing that configuration, (b) moving it to a standard Kubernetes object, and (c) merging multiple Ingress objects into a single config for the load balancer.

The typical software base implementation looks something like what is depicted in Figure 8-1. The Ingress controller is a software system made up of two parts. The first is the Ingress proxy, which is exposed outside the cluster using a service of `type: LoadBalancer`. This proxy sends requests to "upstream" servers. The other component is

the Ingress reconciler, or operator. The Ingress operator is responsible for reading and monitoring Ingress objects in the Kubernetes API and reconfiguring the Ingress proxy to route traffic as specified in the Ingress resource. There are many different Ingress implementations. In some, these two components are combined in a single container; in others, they are distinct components that are deployed separately in the Kubernetes cluster. In Figure 8-1, we introduce one example of an Ingress controller.

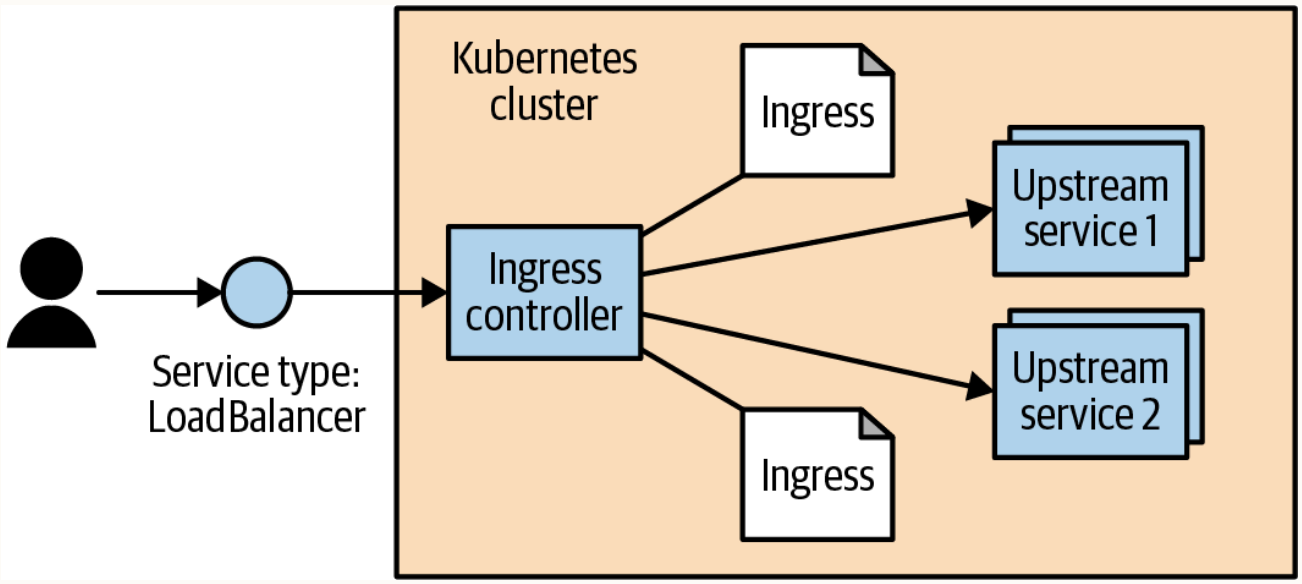


Figure 8-1. The typical software Ingress controller configuration

Ingress Spec Versus Ingress Controllers

While conceptually simple, at an implementation level, Ingress is very different from pretty much every other regular resource object in Kubernetes. Specifically, it is split into a common resource specification and a controller implementation. There is no "standard" Ingress controller that is built into Kubernetes, so the user must install one of many optional implementations.

Users can create and modify Ingress objects just like every other object. But, by default, there is no code running to actually act on those objects. It is up to the users (or the distribution they are using) to install and manage an outside controller. In this way, the controller is pluggable.

There are a couple of reasons that Ingress ended up like this. First of all, there is no one single HTTP load balancer that can be used universally. In addition to many software load balancers (both open source and proprietary), there are also load-balancing capabilities provided by cloud providers (e.g., ELB on AWS), and hardware-based load balancers. The second reason is that the Ingress object was added to Kubernetes before any of the common extensibility capabilities were added (see Chapter 17). As Ingress progresses, it is likely that it will evolve to use these mechanisms.

Installing Contour

While there are many available Ingress controllers, for the examples here, we use an Ingress controller called Contour. This is a controller built to configure the open source (and CNCF project) load balancer called Envoy. Envoy is built to be dynamically configured via an API. The Contour Ingress controller takes care of translating the Ingress objects into something that Envoy can understand.

NOTE

The Contour project was created by Heptio in collaboration with real-world customers and is used in production settings but is now an independent open source project.

You can install Contour with a simple one-line invocation:

```
$ kubectl apply -f https://projectcontour.io/quickstart/contour.yaml
```

Note that this requires execution by a user who has `cluster-admin` permissions.

This one line works for most configurations. It creates a namespace called `projectcontour`. Inside of that namespace it creates a deployment (with two replicas) and an external-facing service of `type: LoadBalancer`. In addition, it sets up the correct permissions via a service account and installs a CustomResourceDefinition (see Chapter 17) for some extended capabilities discussed in "The Future of Ingress".

Because it is a global install, you need to ensure that you have wide admin permissions on the cluster you are installing into. After you install it, you can fetch the external address of Contour via:

```
$ kubectl get -n projectcontour service envoy -o wide
NAME    CLUSTER-IP    EXTERNAL-IP        PORT(S)     ...
contour 10.106.53.14  a477...amazonaws.com 80:30274/TCP ...
```

Look at the `EXTERNAL-IP` column. This can be either an IP address (for GCP and Azure) or a hostname (for AWS). Other clouds and environments may differ. If your Kubernetes cluster doesn't support services of `type: LoadBalancer`, you'll have to change the YAML

for installing Contour to use `type: NodePort` and route traffic to machines on the cluster via a mechanism that works in your configuration.

If you are using `minikube`, you probably won't have anything listed for `EXTERNAL-IP`. To fix this, you need to open a separate terminal window and run `minikube tunnel`. This configures networking routes such that you have unique IP addresses assigned to every service of `type: LoadBalancer`.

Configuring DNS

To make Ingress work well, you need to configure DNS entries to the external address for your load balancer. You can map multiple hostnames to a single external endpoint and the Ingress controller will direct incoming requests to the appropriate upstream service based on that hostname.

For this chapter, we assume that you have a domain called `example.com`. You need to configure two DNS entries: `alpaca.example.com` and `bandicoot.example.com`. If you have an IP address for your external load balancer, you'll want to create A records. If you have a hostname, you'll want to configure CNAME records.

The ExternalDNS project is a cluster add-on that you can use to manage DNS records for you. ExternalDNS monitors your Kubernetes cluster and synchronizes IP addresses for Kubernetes Service resources with an external DNS provider. ExternalDNS supports a wide variety of DNS providers including traditional domain registrars as well as public cloud providers.

Configuring a Local hosts File

If you don't have a domain or if you are using a local solution such as `minikube`, you can set up a local configuration by editing your */etc/hosts* file to add an IP address. You need admin/root privileges on your workstation. The location of the file may differ on your platform, and making it take effect may require extra steps. For example, on Windows the file is usually at *C:\Windows\System32\drivers\etc\hosts*, and for recent versions of macOS, you need to run `sudo killall -HUP mDNSResponder` after changing the file.

Edit the file to add a line like the following:

```
<ip-address> alpaca.example.com bandicoot.example.com
```

For `<ip-address>`, fill in the external IP address for Contour. If all you have is a hostname (like from AWS), you can get an IP address (that may change in the future) by executing `host -t a <address>`.

Don't forget to undo these changes when you are done!

Using Ingress

Now that we have an Ingress controller configured, let's put it through its paces. First, we'll create a few upstream (also sometimes referred to as "backend") services to play with by executing the following commands:

```
$ kubectl create deployment be-default \
  --image=gcr.io/kuar-demo/kuard-amd64:blue \
  --replicas=3 \
  --port=8080
$ kubectl expose deployment be-default
$ kubectl create deployment alpaca \
  --image=gcr.io/kuar-demo/kuard-amd64:green \
  --replicas=3 \
  --port=8080
$ kubectl expose deployment alpaca
$ kubectl create deployment bandicoot \
  --image=gcr.io/kuar-demo/kuard-amd64:purple \
  --replicas=3 \
  --port=8080
$ kubectl expose deployment bandicoot
$ kubectl get services -o wide

NAME         CLUSTER-IP    ... PORT(S)  ... SELECTOR
alpaca       10.115.245.13 ... 8080/TCP ... run=alpaca
bandicoot    10.115.242.3  ... 8080/TCP ... run=bandicoot
be-default   10.115.246.6  ... 8080/TCP ... run=be-default
kubernetes   10.115.240.1  ... 443/TCP  ... <none>
```

Simplest Usage

The simplest way to use Ingress is to have it just blindly pass everything that it sees through to an upstream service. There is limited support for imperative commands to work with Ingress in `kubectl`, so we'll start with a YAML file (see Example 8-1).

Example 8-1. simple-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-ingress
spec:
  defaultBackend:
    service:
      name: alpaca
      port:
        number: 8080
```

Create this Ingress with `kubectl apply`:

```
$ kubectl apply -f simple-ingress.yaml
ingress.extensions/simple-ingress created
```

You can verify that it was set up correctly using `kubectl get` and `kubectl describe`:

```
$ kubectl get ingress
NAME             HOSTS   ADDRESS   PORTS   AGE
simple-ingress   *                 80      13m

$ kubectl describe ingress simple-ingress
Name:             simple-ingress
Namespace:        default
Address:
Default backend:  alpaca:8080
(172.17.0.6:8080,172.17.0.7:8080,172.17.0.8:8080)
Rules:
  Host  Path  Backends
  ----  ----  --------
  *     *     alpaca:8080 (172.17.0.6:8080,172.17.0.7:8080,172.17.0.8:8080)
Annotations:
  ...

Events:  <none>
```

This sets things up so that *any* HTTP request that hits the Ingress controller is forwarded on to the `alpaca` service. You can now access the `alpaca` instance of `kuard` on any of

the raw IPs/CNAMEs of the service; in this case, either `alpaca.example.com` or `bandicoot.example.com`. This doesn't, at this point, add much value over a simple service of `type: LoadBalancer`. The following sections experiment with more complex configurations.

Using Hostnames

Things start to get interesting when we direct traffic based on properties of the request. The most common example of this is to have the Ingress system look at the HTTP host header (which is set to the DNS domain in the original URL) and direct traffic based on that header. Let's add *another* Ingress object for directing traffic to the `alpaca` service for any traffic directed to `alpaca.example.com` (see Example 8-2).

Example 8-2. host-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-ingress
spec:
  defaultBackend:
    service:
      name: be-default
      port:
        number: 8080
  rules:
  - host: alpaca.example.com
    http:
      paths:
      - pathType: Prefix
        path: /
        backend:
          service:
            name: alpaca
            port:
              number: 8080
```

Create this Ingress with `kubectl apply`:

```
$ kubectl apply -f host-ingress.yaml
ingress.extensions/host-ingress created
```

We can verify that things are set up correctly as follows:

```
$ kubectl get ingress
NAME           HOSTS              ADDRESS   PORTS   AGE
host-ingress   alpaca.example.com           80      54s
simple-ingress *                            80      13m

$ kubectl describe ingress host-ingress
Name:           host-ingress
Namespace:      default
Address:
Default backend:  be-default:8080 (<none>)
Rules:
  Host            Path  Backends
  ----            ----  --------
  alpaca.example.com
                  /   alpaca:8080 (<none>)
Annotations:
  ...

Events:  <none>
```

There are a couple of confusing things here. First, there is a reference to the `default-http-backend`. This is a convention that only some Ingress controllers use to handle requests that aren't handled in any other way. These controllers send those requests to a service called `default-http-backend` in the `kube-system` namespace. This convention is surfaced client-side in `kubectl`. Next, there are no endpoints listed for the `alpaca` backend service. This is a bug in `kubectl` that is fixed in Kubernetes v1.14.

Regardless, you should now be able to address the `alpaca` service via *http://alpaca.example.com*. If instead you reach the service endpoint via other methods, you should get the default service.

Using Paths

The next interesting scenario is to direct traffic based on not just the hostname, but also the path in the HTTP request. We can do this easily by specifying a path in the `paths` entry (see Example 8-3). In this example, we direct everything coming into *http://bandicoot.example.com* to the `bandicoot` service, but we also send *http://bandicoot.example.com/a* to the `alpaca` service. This type of scenario can be used to host multiple services on different paths of a single domain.

Example 8-3. path-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-ingress
spec:
  rules:
  - host: bandicoot.example.com
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: bandicoot
            port:
              number: 8080
      - pathType: Prefix
        path: "/a/"
        backend:
          service:
            name: alpaca
            port:
              number: 8080
```

When there are multiple paths on the same host listed in the Ingress system, the longest prefix matches. So, in this example, traffic starting with `/a/` is forwarded to the `alpaca` service, while all other traffic (starting with `/`) is directed to the `bandicoot` service.

As requests get proxied to the upstream service, the path remains unmodified. That means a request to `bandicoot.example.com/a/` shows up to the upstream server that is configured for that request hostname and path. The upstream service needs to be ready to serve traffic on that subpath. In this case, `kuard` has special code for testing, where it responds on the root path (`/`) along with a predefined set of subpaths (`/a/`, `/b/`, and `/c/`).

Cleanup

To clean up, execute the following:

```
$ kubectl delete ingress host-ingress path-ingress simple-ingress
$ kubectl delete service alpaca bandicoot be-default
$ kubectl delete deployment alpaca bandicoot be-default
```

Advanced Ingress Topics and Gotchas

Ingress supports some other fancy features. The level of support for these features differs based on the Ingress controller implementation, and two controllers may implement a feature in slightly different ways.

Many of the extended features are exposed via annotations on the Ingress object. Be careful; these annotations can be hard to validate and are easy to get wrong. Many of these annotations apply to the entire Ingress object and so can be more general than you might like. To scope the annotations down, you can always split a single Ingress object into multiple Ingress objects. The Ingress controller should read them and merge them together.

Running Multiple Ingress Controllers

There are multiple Ingress controller implementations, and you may want to run multiple Ingress controllers on a single cluster. To solve this case, the IngressClass resource exists so that an Ingress resource can request a particular implementation. When you create an Ingress resource, you use the `spec.ingressClassName` field to specify the specific Ingress resource.

NOTE

In Kubernetes prior to version 1.18, the `IngressClassName` field did not exist and the `kubernetes.io/ingress.class` annotation was used instead. While this is still supported by many controllers, it is recommended that people move away from the annotation as it will likely be deprecated by controllers in the future.

If the `spec.ingressClassName` annotation is missing, a default Ingress controller is used. It is specified by adding the `ingressclass.kubernetes.io/is-default-class` annotation to the correct IngressClass resource.

Multiple Ingress Objects

If you specify multiple Ingress objects, the Ingress controllers should read them all and try to merge them into a coherent configuration. However, if you specify duplicate and conflicting configurations, the behavior is undefined. It is likely that different Ingress controllers will behave differently. Even a single implementation may do different things depending on nonobvious factors.

Ingress and Namespaces

Ingress interacts with namespaces in some nonobvious ways. First, due to an abundance of security caution, an Ingress object can refer to only an upstream service in the same namespace. This means that you can't use an Ingress object to point a subpath to a service in another namespace.

However, multiple Ingress objects in different namespaces can specify subpaths for the same host. These Ingress objects are then merged to come up with the final config for the Ingress controller.

This cross-namespace behavior means that coordinating Ingress globally across the cluster is necessary. If not coordinated carefully, an Ingress object in one namespace could cause problems (and undefined behavior) in other namespaces.

Typically there are no restrictions built into the Ingress controller around which namespaces are allowed to specify which hostnames and paths. Advanced users may try to enforce a policy for this using a custom admission controller. There are also evolutions of Ingress described in "The Future of Ingress" that address this problem.

Path Rewriting

Some Ingress controller implementations support, optionally, doing path rewriting. This can be used to modify the path in the HTTP request as it gets proxied. This is usually specified by an annotation on the Ingress object and applies to all requests that are specified by that object. For example, if we were using the NGINX Ingress controller, we could specify an annotation of `nginx.ingress.kubernetes.io/rewrite-target: /`. This can sometimes make upstream services work on a subpath even if they weren't built to do so.

There are multiple implementations that not only implement path rewriting, but also support regular expressions when specifying the path. For example, the NGINX con-

troller allows regular expressions to capture parts of the path and then use that captured content when doing rewriting. How this is done (and what variant of regular expressions is used) is implementation-specific.

Path rewriting isn't a silver bullet, though, and can often lead to bugs. Many web applications assume that they can link within themselves using absolute paths. In that case, the app in question may be hosted on `/subpath` but have requests show up to it on `/`. It may then send a user to `/app-path`. There is then the question of whether that is an "internal" link for the app (in which case it should instead be `/subpath/app-path`) or a link to some other app. For this reason, it is probably best to avoid subpaths for any complicated applications if you can help it.

Serving TLS

When serving websites, it is becoming increasingly necessary to do so securely using TLS and HTTPS. Ingress supports this (as do most Ingress controllers).

First, users need to specify a Secret with their TLS certificate and keys—something like what is outlined in Example 8-4. You can also create a Secret imperatively with `kubectl create secret tls <secret-name> --cert <certificate-pem-file> --key <private-key-pem-file>`.

Example 8-4. tls-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  creationTimestamp: null
  name: tls-secret-name
type: kubernetes.io/tls
data:
  tls.crt: <base64 encoded certificate>
  tls.key: <base64 encoded private key>
```

Once you have the certificate uploaded, you can reference it in an Ingress object. This specifies a list of certificates along with the hostnames that those certificates should be used for (see Example 8-5). Again, if multiple Ingress objects specify certificates for the same hostname, the behavior is undefined.

Example 8-5. tls-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  tls:
  - hosts:
    - alpaca.example.com
    secretName: tls-secret-name
  rules:
  - host: alpaca.example.com
    http:
      paths:
      - backend:
          serviceName: alpaca
          servicePort: 8080
```

Uploading and managing TLS secrets can be difficult. In addition, certificates can often come at a significant cost. To help solve this problem, there is a nonprofit called "Let's Encrypt" running a free Certificate Authority that is API-driven. Since it is API-driven, it is possible to set up a Kubernetes cluster that automatically fetches and installs TLS certificates for you. It can be tricky to set up, but when working, it's very simple to use. The missing piece is an open source project called cert-manager created by Jetstack, a UK startup, onboarded to the CNCF. The *cert-manager.io* website or GitHub repository has details on how to install cert-manager and get started.

Alternate Ingress Implementations

There are many different implementations of Ingress controllers, each building on the base Ingress object with unique features. It is a vibrant ecosystem.

First, each cloud provider has an Ingress implementation that exposes the specific cloud-based L7 load balancer for that cloud. Instead of configuring a software load balancer running in a Pod, these controllers take Ingress objects and use them to configure, via an API, the cloud-based load balancers. This reduces the load on the cluster and the management burden for the operators, but can often come at a cost.

The most popular generic Ingress controller is probably the open source NGINX Ingress controller. Be aware that there is also a commercial controller based on the proprietary

NGINX Plus. The open source controller essentially reads Ingress objects and merges them into an NGINX configuration file. It then signals to the NGINX process to restart with the new configuration (while responsibly serving existing in-flight connections). The open source NGINX controller has an enormous number of features and options exposed via annotations.

Emissary and Gloo are two other Envoy-based Ingress controllers that are focused on being API gateways.

Traefik is a reverse proxy implemented in Go that also can function as an Ingress controller. It has a set of features and dashboards that are very developer-friendly.

This just scratches the surface. The Ingress ecosystem is very active, and there are many new projects and commercial offerings that build on the humble Ingress object in unique ways.

The Future of Ingress

As you have seen, the Ingress object provides a very useful abstraction for configuring L7 load balancers—but it hasn't scaled to all the features that users want and various implementations are looking to offer. Many of the features in Ingress are underdefined. Implementations can surface these features in different ways, reducing the portability of configurations between implementations.

Another problem is that it is easy to misconfigure Ingress. The way that multiple objects compbine opens the door for conflicts that are resolved differently by different implementations. In addition, the way that these are merged across namespaces breaks the idea of namespace isolation.

Ingress was also created before the idea of a service mesh (exemplified by projects such as Istio and Linkerd) was well known. The intersection of Ingress and service meshes is still being defined. Service meshes are covered in greater detail in Chapter 15.

The future of HTTP load balancing for Kubernetes looks to be the *Gateway* API, which is in the midst of development by the Kubernetes special interest group (SIG) dedicated to networking. The Gateway API project is intended to develop a more modern API for routing in Kubernetes. Though it is more focused on HTTP balancing, Gateway also includes resources for controlling Layer 4 (TCP) balancing. The Gateway APIs are still very much under development, so it is strongly recommended that people stick to the existing

Ingress and Service resources that are currently present in Kubernetes. The current state of the Gateway API can be found online.

Summary

Ingress is a unique system in Kubernetes. It is simply a schema, and the implementations of a controller for that schema must be installed and managed separately. But it is also a critical system for exposing services to users in a practical and cost-efficient way. As Kubernetes continues to mature, expect to see Ingress become more and more relevant.

1 The Open Systems Interconnection (OSI) model is a standard way to describe how different networking layers build on each other. TCP and UDP are considered to be Layer 4, while HTTP is Layer 7.

Previous chapter
< 7. Service Discovery

Next chapter
9. ReplicaSets >